

From text to tokens.

Learn what comes next.

Inspect how language becomes model input,
then follow the probabilities behind generated text.

🕒 20-25 min core reading · 20-30 min practice · 12 short sections

“Find a light movie” can mean cheerful, short, or easy to follow. Language systems must represent the text and use context to interpret it. This chapter follows the path from text to tokens, from tokens to predictions, and from predictions to generated text.

WHAT YOU WILL UNDERSTAND

The main question: how does a language model turn a sequence of symbols into a distribution over what comes next?

The practical path: inspect a tokenizer, count movie-word transitions, calculate prediction loss, and compare generation policies.

Reading path: twelve short sections, three diagrams, two interactive experiments, and a Python workbook. Further details stay in expandable notes.

1. Start with the language task

Natural language processing (NLP) studies computational methods for human language. It includes recognizing structure and meaning, extracting information, and producing text. A language model is one tool within NLP; it is not the definition of the whole field.

Movie application	Task	What a useful output looks like
"The ending was wonderful."	Sentiment classification	A label or score, assessed against a defined annotation policy
"Find an adventure under 120 minutes."	Information extraction	Genre = adventure; maximum runtime = 120
"What did reviewers like about it?"	Summarization / question answering	A concise answer supported by the supplied reviews
"this movie is ..."	Language modeling	A probability distribution over the next token

Syntax concerns how expressions are structured; **semantics** concerns meaning; **pragmatics** concerns meaning in a situation. "Not a bad movie" illustrates why individual words are insufficient. A practical system needs a clear input/output contract, suitable examples, and task-specific evaluation-not just fluent text.

2. Decide what counts as a token

A **token** is a unit the model consumes or predicts. It may be a word, punctuation mark, word fragment, character, or byte-based piece. A **vocabulary** maps allowed pieces to token IDs. Token boundaries depend on the tokenizer; one word is not necessarily one token.

Word tokenization is easy to read but struggles with names, spelling variations, and unseen words. Character tokenization uses smaller units but longer sequences. **Subword tokenization** balances reuse and sequence length: a rare word may be represented by familiar smaller pieces. A tokenizer with all 256 byte values available can represent any valid UTF-8 input through those bytes, though individual tokens need not decode to complete characters.

Preprocessing changes the information available. Lowercasing merges "US" and "us"; removing punctuation can lose sentence structure; deleting "not" reverses the meaning of a review. Use the preprocessing expected by the model instead of applying generic text cleaning automatically.

▼ Go deeper: normalization, stemming, and lemmatization

Normalization makes chosen surface forms consistent, for example a Unicode normalization policy or standardized line endings. It can be useful, but it is not always reversible. Keep original text when exact quotations or offsets matter.

Stemming applies rules to shorten words, sometimes producing a fragment. **Lemmatization** maps inflected forms to a dictionary form using linguistic information. They can help a particular search pipeline, but are not mandatory preparation for a pretrained language model. Likewise, stop-word removal may suit some retrieval designs while damaging negation or grammatical context in another task. Measure the effect on the actual task.

3. Learn how byte-pair encoding builds reusable pieces

Byte-pair encoding (BPE) repeatedly merges a frequent adjacent pair of existing units into a new unit. A training phase learns an ordered merge list; encoding new text then applies those learned rules. It does not relearn a vocabulary for every question. The [original subword BPE paper](#) explains the motivation for rare-word handling.

Our teaching version starts with 256 UTF-8 byte tokens. On the six movie sentences, the first winning pair is byte 115 ("s") followed by byte 32 (space), observed 10 times. It becomes ID 256. The next merge combines byte 105 ("i") with ID 256 to form `isSP`, ID 257. Here `SP` is only a display symbol for a space.

With 16 merges, `this movie is fun.` becomes four pieces: `thisSPmovieSPisSP`, `f`, `un`, and `.`. Their IDs are **270, 102, 268, 46**. Concatenating their underlying bytes reconstructs the original text. The joined phrase is possible because this simple tokenizer allows merges across spaces; many production tokenizers restrict merging within pre-tokenized regions.

▼ Go deeper: a precise tokenizer contract

The workbench counts adjacent pairs separately within each training sentence, including overlapping occurrences. It chooses the greatest count, breaking ties by smaller left ID and then smaller right ID. Replacements run left to right without overlap. It stops at the requested merge budget or when no pair occurs twice. New IDs start at 256; encoding applies the merge list in training order. All base bytes remain available.

Real tokenizer families differ. BPE learns merges; **WordPiece** uses a different vocabulary-building procedure; **Unigram tokenization** selects pieces using a probabilistic vocabulary model. SentencePiece is a toolkit that supports methods including BPE and Unigram; it is not a synonym for one algorithm. See the [SentencePiece paper](#). Always pair the tokenizer files, normalization settings, special-token rules, and vocabulary version with the model that expects them.

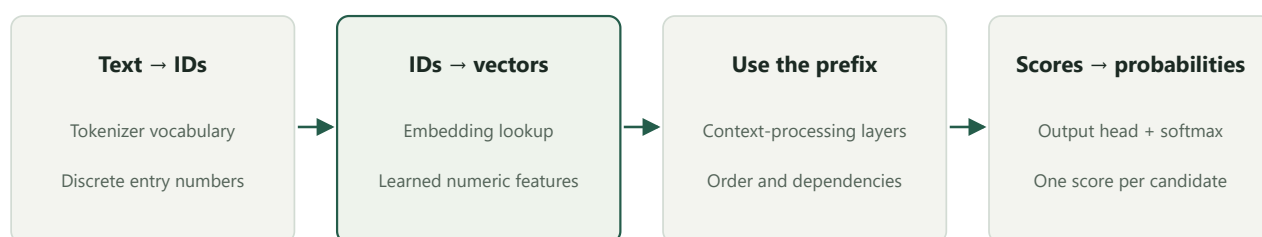
4. Separate token IDs from learned representations

ID 270 is a lookup key, not a meaning score. The difference between IDs 270 and 102 says nothing about semantic similarity. In a neural model, an **embedding table** stores a learned vector for each ID. A vocabulary of 1,000 entries with embedding dimension 64 has 64,000 embedding values. Looking up five tokens produces five vectors of length 64.

Context-processing layers combine those vectors. A token's contextual representation can differ between "a light comedy" and "a light above the stage." Order must also be represented, through recurrence, positional information, or another architecture mechanism. Tokenization alone does not provide comprehension.

Special tokens have designated roles. A beginning marker can start a sequence; an end marker can tell a generator to stop; padding fills unused batch positions. A padding mask prevents placeholders from acting as real content. The exact symbols and insertion rules are model-specific: typing a marker's visible spelling is not universally equivalent to inserting its special ID.

➤ Tokenization chooses pieces; the model learns representations



The tokenizer and the model must agree on the vocabulary and IDs.

This is a neural language-model pipeline. The workbench's bigram estimates probabilities from counts.

Figure 1. A tokenizer maps text to vocabulary IDs. An embedding lookup gives each ID a learned vector. Context-processing layers combine those vectors; an output head produces one logit per candidate token. Token IDs themselves are not semantic coordinates.

▼ Go deeper: Unicode, bytes, and context budgets

A displayed character can contain multiple Unicode code points, and a code point can occupy several UTF-8 bytes. The clapperboard emoji 🎬 is one code point and four UTF-8 bytes. This tokenizer's ASCII training corpus does not merge those bytes, so the emoji uses four tokens. Some byte fragments are invalid text on their own; the explorer displays their hexadecimal values and decodes only after concatenation.

A **context window** limits the token positions a model can process under its supported configuration. Input, formatting markers, and generated output consume capacity according to that system's rules. Token counts vary with language, whitespace, numbers, and the tokenizer. There is no reliable universal "one token equals one word" conversion. A longer window increases possible access, not a guarantee that every detail will be used well.

5. Define a language model as a probability model

A language model assigns probabilities to token sequences. The chain rule decomposes a sequence into next-token predictions:

$$P(x_1, \dots, x_n) = \prod_{t=1}^n P(x_t \mid x_1, \dots, x_{t-1}).$$

An **autoregressive** model predicts using the earlier prefix and then can extend it one token at a time. A **unigram** model ignores previous tokens. A **bigram** model uses only the immediately previous token. More generally, an **n-gram** uses at most the previous $n - 1$ tokens. These are explicit context limitations, not claims that language itself has such a short memory. D2L's [language-model chapter](#) develops this sequence view.

Our bigram is fitted by counting lowercase words and punctuation in six authored sentences. It has 13 possible target tokens, including an end token and an unknown token. The beginning marker is a context, never a target. This is a separate word tokenizer; it does not use the BPE IDs from the first experiment.

After `is`, the corpus has `fun` twice, `slow` once, and `warm` once. The unsmoothed estimate is $P(\text{fun} \mid \text{is}) = 2/4 = 0.5$. Both "this movie is" and "this film is" produce the same next-token distribution in this model because their last token is identical.

6. Handle unseen transitions and define training targets

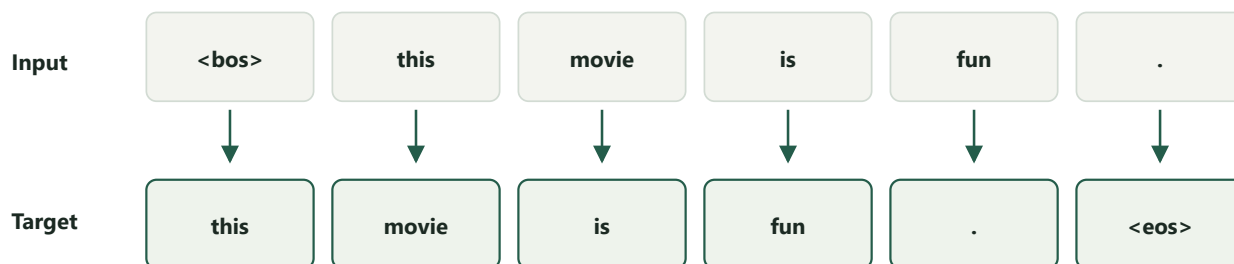
Observed counts alone assign zero probability to unseen transitions. **Additive smoothing** adds a nonnegative value α to each possible target count:

$$P(w \mid h) = \frac{C(h, w) + \alpha}{C(h) + \alpha V}.$$

h is the previous token, $C(h, w)$ the pair count, $C(h)$ the number of observed continuations, and V the target-vocabulary size. With $\alpha = 1$, our example gives $(2 + 1)/(4 + 13) = 3/17 \approx 0.1765$. The remaining probability has been distributed across other candidates. Smoothing avoids zero probabilities here; it does not recover missing knowledge or necessarily improve every score. With $\alpha = 0$ and no observations for a context, our generator reports that no continuation is available.

Neural language models learn parameters instead of storing only a count table. **Self-supervised learning** obtains targets from the text itself: each observed next token is a training label. In **teacher forcing**, training supplies the observed prefix rather than the model's previous sampled guesses. The inputs and targets are shifted by one position.

➤ Shift the targets by one position



Six scored targets. No padding in this example. The tiny bigram uses only the immediately previous token.

Figure 2. Each input prefix predicts the next observed token. The beginning marker supplies the first context; the end marker is a prediction target. A causal model cannot use future targets to predict an earlier position.

At each position, the output head produces **logits**: unrestricted scores over the vocabulary. Softmax converts them to probabilities. Cross-entropy for the correct target is $-\ln p_{\text{target}}$; backpropagation adjusts parameters to reduce the average loss. Seeing the target through the input or future context would make the training problem invalid. A causal architecture must block that access even when many training positions are processed together.

7. Interpret loss and perplexity carefully

A target given probability 0.5 contributes $-\ln(0.5) \approx 0.6931$ **nats** of loss; probability 0.1 contributes about 2.3026. Nats use the natural logarithm. Higher probability on the observed target gives lower loss.

For N scored targets, mean negative log-probability is $L = -\frac{1}{N} \sum_t \ln p_t$, and **perplexity** is $\exp(L)$. Equivalently, it is the inverse geometric mean of the observed-target probabilities. A uniform distribution over V candidates has perplexity V ; a general perplexity is not literally the number of choices at each position.

Our familiar sentence `this movie is fun .` has six targets when the end token is included. Without smoothing their probability product is $1/8$, so $L = \ln(8)/6 \approx 0.3466$ and perplexity is **1.4142**. With add-one smoothing, its perplexity becomes **4.2178**. This visible training sentence cannot establish which policy generalizes better.

▼ Go deeper: fair perplexity comparisons and unknown words

Compare perplexities only under compatible tokenization, text, context, masking, and boundary policies. Different token units change the number and difficulty of predictions. For fixed-window models, chunking can discard useful context; a sliding evaluation window needs a clear rule for which positions are scored. See the [Hugging Face perplexity guide](#).

In our word model, `dazzling` maps to the unknown symbol because it is outside the vocabulary. With no smoothing, two transitions in that mapped sentence have probability zero, so loss and perplexity are infinite. JSON reports `null` for these infinite metrics and supplies the zero-probability count. With smoothing the score is finite, but it is still the probability of the **mapped token sequence**, not of the exact unseen spelling. The byte tokenizer has no such unknown-token fallback because every input byte has an entry.

8. Generate text by choosing and feeding back tokens

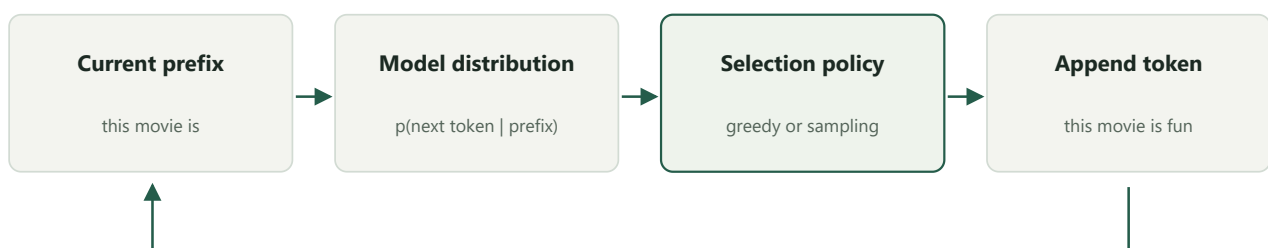
Generation repeatedly computes a distribution, selects a token, and appends it to the prefix. It ends at a stop condition, such as an end token or an output limit. During generation, an earlier mistake can become part of the next input.

Greedy decoding chooses the highest-probability token at each step. It need not find the most probable complete sequence. **Sampling** draws from a distribution, allowing alternatives. A random seed makes a run reproducible under the same implementation; it does not make the output more correct.

Temperature $T > 0$ rescales logits before softmax: $q_i \propto \exp(z_i/T)$. Lower temperature sharpens differences; higher temperature flattens them. Given base probabilities directly, the equivalent rule is $q_i \propto p_i^{1/T}$. Positive temperature alone preserves the ranking, so greedy choices do not change unless other rules or numerical details intervene.

Top-k sampling keeps the k highest-scoring candidates and renormalizes. **Top-p**, or nucleus sampling, keeps a smallest high-probability prefix of the sorted candidates whose cumulative mass reaches a chosen threshold. These are decoding policies, not changes to learned weights. The workbench implements temperature and top-k; top-p is explained only. [Generation parameter reference](#).

➤ Generation repeatedly feeds back the chosen token



Repeat with the extended prefix

During generation the model sees its own earlier choices; training uses observed target prefixes.

Figure 3. A model produces a next-token distribution, a decoding policy chooses one token, and the new token extends the prefix. Repeat until an end token or a length limit. Choosing a likely token does not check whether a claim is true.

9. Distinguish language likelihood from task success

A frequent phrase can be false. A rare name can be correct. A next-token probability measures a model's continuation preference under its inputs and training, not the probability that a whole claim is true. Raising temperature does not add evidence; lowering it does not verify a streaming-availability claim.

Choose evaluation for the application: extraction needs field-level checks; a review classifier needs classification metrics; a grounded answer needs evidence and citation checks. Keep independent evaluation text separate from tokenizer/model fitting and from repeated tuning. Duplicated reviews, related documents, or future data crossing a split can inflate apparent quality.

▼ Go deeper: causal prediction, masked prediction, and instruction following

A **causal language-model objective** predicts each next token from its earlier context. A **masked language-model objective** predicts selected hidden or corrupted tokens using the permitted surrounding context, as illustrated in the [BERT paper](#). Their conditioning differs, so ordinary next-token perplexity does not directly apply to a masked model. Encoder-decoder objectives can condition an output sequence on another input sequence, such as a review to summarize.

Learning to predict text supplies a useful training signal, but a base model is not automatically a reliable instruction-following assistant. Later adaptation, preference training, retrieval, tools, and application checks address different needs. Those topics build on this foundation in later chapters. A successful tokenizer round trip or low training loss is evidence about a specific mechanism, not mastery of the user's task.

10. Explore the text-to-prediction workbench

Start by encoding the same text with 0, 8, 16, and 24 merges. Compare a leading space, an uppercase letter, and an emoji. Then switch to the language model: compare greedy and sampled continuations, change smoothing, and inspect a fixed sentence's loss. Notice that temperature changes selection probabilities while the base-model score stays fixed.

Inspect the pieces. Predict the next one.

Two independent experiments: a learned byte-pair tokenizer and a word-based bigram model. The second model does not use the first experiment's token IDs.

Tokenization: "this movie is fun." has 18 code points and 18 UTF-8 bytes. With 16 learned merges it becomes four tokens: `thisSPmovieSPisSP` | `f` | `un` | `.`, IDs `270, 102, 268, 46`. The first learned pair is byte 115 ("s") + byte 32 (space), observed 10 times. The full decoded text is unchanged.

Next token after "is": counts are fun = 2, slow = 1, warm = 1. With $\alpha = 0$ their probabilities are 0.5, 0.25, 0.25. With $\alpha = 1$ and vocabulary size 13, they become 3/17, 2/17, 2/17; every other target gets 1/17. Greedy continuation of "this movie is" is "fun." followed by the end token.

Fixed sentence / policy	Mean loss	Perplexity
this movie is fun . $\alpha = 0$	0.3466	1.4142
this movie is fun . $\alpha = 1$	1.4393	4.2178
this movie is dazzling . $\alpha = 0$	Infinite	Infinite

All scores include six targets, including the end token. "dazzling" maps to the unknown symbol. Scoring uses base probabilities, independent of temperature and sampling. These are teaching cases, not a test-set benchmark or factual-confidence measurement.

The first experiment fits BPE merges; the second fits a bigram count model. Neither is a neural LLM. The corpus is intentionally tiny and visible, including one repeated sentence that changes the counts. All displayed quality measurements are teaching calculations, not held-out benchmark results.

The four-file Python workbook contains `data.json`, `language.py`, `test_language.py`, and `README.md`. It needs Python 3.10+ and the standard library, without model downloads or network calls.

```
py language.py tokenize --text "this movie is fun." --merges 16
py language.py model --method greedy
py language.py model --method sample --temperature 0.5 --top-k 3
py language.py model --alpha 0 --score-case unknown
py -m unittest -v test_language.py
```

Use `python` or `python3` if needed. Commands print JSON and do not save a model. Invalid CLI arguments exit with code 2. The report includes token pieces, merge rules, candidate probabilities, generated steps, and scoring traces according to the chosen command.

Exercise 1: follow a smoothed probability

After `is`, counts are fun = 2, slow = 1, warm = 1, with 13 possible targets. What are the probabilities of `fun` and an unobserved target when $\alpha = 1$? Why is the denominator not 5?

▼ Reveal the reasoning

The total is $4 + 13 = 17$ because one is added to every candidate count. fun gets $3/17$; an unobserved target gets $1/17$. Adding only one to the denominator would not normalize the 13 adjusted counts.

Exercise 2: calculate perplexity from two targets

Two target probabilities are 0.5 and 0.25. Find the mean natural-log loss and perplexity. Would changing the generation temperature change this base-model score?

▼ Reveal the reasoning

Mean loss = $-(\ln 0.5 + \ln 0.25)/2 \approx 1.0397$. Perplexity = $\exp(1.0397) \approx 2.8284$, the inverse square root of 0.5×0.25 . The score stays the same when only decoding changes. Scoring a temperature-adjusted distribution would be a different, explicitly defined evaluation.

Exercise 3: diagnose a misleading comparison

One model gets perplexity 8 on word tokens; another gets 4 on byte-based tokens. The second also generates "this movie is fun." Can you conclude it is the better movie assistant?

▼ Reveal the reasoning

No. The token units and scoring conditions differ, and a fluent sentence does not establish factual accuracy or personalized usefulness. Compare on a shared task and appropriate independent examples, while recording tokenizer and scoring policies. Check whether catalog claims are supported and whether recommendations satisfy the viewer's constraints.

11. Check your understanding

01 What does token ID 270 mean in the byte-pair experiment?

- A. It identifies one entry in this particular tokenizer vocabulary.
- B. It is a universal score for how meaningful the token is.
- C. It must refer to the same text in every language model.

Answer: A. An ID is an index. Its mapping depends on the tokenizer vocabulary and version; learned embeddings supply numerical representations.

02 After “is”, fun occurs twice in four observations. With $\alpha = 1$ and 13 possible targets, what is $p(\text{fun} \mid \text{is})$?

- A. $2/4$
- B. $3/17$
- C. $3/5$

Answer: B. Add one to the candidate count and one to each of 13 target counts: $(2 + 1) / (4 + 13) = 3/17$.

03 Which information may predict “fun” in a causal language-model training example?

- A. The “fun” target itself and every later word.
- B. Only the document title, never earlier words.
- C. The tokens preceding “fun”, without access to future targets.

Answer: C. Training supplies the observed prefix, but the target at a position must not leak into that position’s prediction. A causal mask enforces this in a transformer.

04 What does raising temperature change in the workbench?

- A. The stored corpus counts.
- B. The distribution used for token selection; fixed-sentence base-model perplexity stays unchanged.
- C. The truth of the generated statement.

Answer: B. Temperature changes decoding probabilities, not learned counts or factual verification. The score panel explicitly uses the unmodified base distribution.

05 A model has lower perplexity on a visible training sentence. What follows?

- A. Its answers are guaranteed more truthful.
- B. It is better than every model with a different tokenizer.
- C. It assigns that token sequence a higher geometric-mean target probability; independent evaluation is still needed.

Answer: C. Perplexity summarizes token prediction under a specified tokenizer, context, and scoring policy. It is not factual accuracy, and a training example is not a held-out benchmark.

12. Connect language modeling to attention

You now have the full chain: **text** → **token IDs** → **representations** → **next-token scores** → **a selection rule**. Tokenization chooses the units, the model estimates probabilities, and decoding chooses how to continue. Keep those responsibilities distinct when debugging.

The bigram's weakness is concrete: it cannot distinguish two prefixes ending in the same token. **Chapter 24: Attention and transformers explained** will show how a neural architecture can combine information from many permitted positions rather than relying on one previous token or a single recurrent state.

▼ Quick-reference glossary

Term	Meaning
NLP / language model	The broader language field / a probability model over token sequences
Token / vocabulary / ID	A model unit / the allowed entries / an index identifying one entry
BPE / byte fallback	Learned pair merges / using basic bytes when larger pieces do not fit
Embedding / context	A learned vector representation / information available for a prediction
Bigram / autoregressive	Conditioning on one previous token / predicting from an earlier prefix
Smoothing / temperature	Adjusting probability estimates / reshaping probabilities for decoding
Logit / softmax	An unrestricted candidate score / conversion of scores to probabilities
Teacher forcing / causal mask	Training with observed prefixes / preventing access to future positions
Cross-entropy / perplexity	Target prediction loss / exponential of mean natural-log loss
Greedy / top-k / top-p	Best next token / fixed candidate count / cumulative-mass candidate selection

Sources and further reading

Primary references are linked beside the relevant concepts: the BPE, SentencePiece, and BERT papers; D2L's language-model chapter; and Hugging Face's perplexity and generation documentation. The corpus, numerical examples, diagrams, and workbook are original teaching material. No commercial tokenizer, pretrained LLM, or external generation service is used by the experiments.